Enhancing Research Workflow with Python-Based Automation for Data Management

Kai Spicer, Jacob Anderson, Matt Asmussen

Energy and Environment Directorate, Pacific Northwest National Laboratory, Richland, Washington, USA

Abstract

Processes requiring manual data transfer are time-consuming and can lead to human errors such as incorrect formatting, transcription mistakes, and data mismanagement. By automating these tasks, researchers can reduce the likelihood of error while saving time and resources. As data-driven decisions become increasingly central to research, the need to collect and distribute data at larger scales has grown significantly, often resulting in considerable time and resources spent on manual data handling. To address this, I collaborated with the Radiological Materials team to develop automated Python scripts designed to streamline data transformation processes. Specifically, the generation of eSTOMP cards was automated to assist in the creation of simulations analyzing the long-term disposal of waste forms. To enhance usability, this application was paired with a user-friendly graphical user interface (GUI), making the tool accessible to researchers without extensive coding experience. Additionally, the application integrates with a growing database of real-world nuclear waste form data, enabling easy access to empirical information for use in creating models. This application not only increases efficiency by saving time but also enhances accuracy by eliminating human errors associated with manual data transfer. The result of this work is the ability to create more models with greater variation, allowing for a deeper understanding of the underlying system. Ultimately, this application enables the team to dedicate more time to testing, modeling and analyzing a wider variety of waste forms while minimizing time spent on data management tasks.

Intro

The Radiological Materials Group at the Pacific Northwest National Laboratory (PNNL) is focused on understanding and developing treatment plans for the variety of wastes that are found at the Hanford site. Over 30 years of plutonium production the Hanford Site generated vast quantities of radioactive waste, with more than 56 million gallons currently stored in underground tanks. These tanks contain unique and complex waste due to the different plutonium reprocessing methods used and mixing of waste between tanks over time. To process this waste, they first take it out of the tanks and vitrify or grout it depending on the categorization of the waste. The specific team I was supporting is focused on modeling what happens when the waste forms are placed in the Integrated Disposal Facility (IDF) where their disposal performance over thousands of years must be modeled [2]. One of the main tools we use to understand the interactions within the IDF is eSTOMP (extreme-scale Subsurface Transport Over Multiple Phases). This simulation software was developed in the early 2000's by researchers at PNNL and can be used to simulate highly complex processes like multifluid flow, heat transport and biogeochemistry. For these complex calculations to be run efficiently eSTOMP uses parallel process and must be run using high performance computing (HPC). Due to the many starting conditions needed to run a simulation, an input card is required that organizes all the assumptions and variables that the modeler specifies. These cards are organized so that they can be easily understood by simulation software, but this means that they look complex to the human eye [1]. Figure 1 shows a snapshot of one of these input cards.

```
~Rock/Soil Zonation Card
formatted zonation file,grout.zon,
HD_BACKF,
LD_BACKF,
W1,
~Mechanical Properties Card
HD_BACKF,2.710,g/cm^3,0.35,0.35,,,Constant       ,1,
LD_BACKF,2.710,g/cm^3,0.37,0.37,,,Constant ,1,
W1,2.820,g/cm^3,0.557,0.557,,,Constant ,1,
~Hydraulic Properties Card
HD_BACKF,4.91e-03,hc cm/s,4.91e-03,hc cm/s,4.91e-03,hc cm/s,
LD_BACKF,1.86e-02,hc cm/s,1.86e-02,hc cm/s,1.86e-02,hc cm/s,
W1,1.54E-09,hc cm/s,1.54E-09,hc cm/s,1.54E-09,hc cm/s,
~Saturation Function Card
HD_BACKF,van Genuchten,6.50e-02,1/cm,1.7,0.086,,
LD_BACKF,van Genuchten,5.70e-02,1/cm,2.8,0.081,,
W1,van Genuchten,6.03E-06,1/cm,1.649,0.108,,
~Aqueous Relative Permeability Card
HD_BACKF,Mualem,,
LD_BACKF,Mualem,,
W1,Mualem,,
~Solute/Porous Media Interactions Card
HD_BACKF,0.0,cm, 0.,cm,
LD_BACKF,0.0,cm, 0.,cm,
W1,0.0,cm, 0.,cm,
~Aqueous Species Card
1,Conventional,0.315,cm^2/yr,Constant,1.0,
CO2(aq),
~Solid Species Card
4,
calcite,2.710,g/cm^3,100.0873,g/mol,
solution,1.7,g/cm^3,18.01534,g/mol,
CO2_product,1.7,g/cm^3,44.009,g/mol,
solution_product,1.7,g/cm^3,44.009,g/mol,
~Conservation Equations Card
3,
Total_calcite,3,calcite,1,solution_product,1,CO2_product,1,
Total_CO2(aq),2,CO2(aq),1,CO2_product,1,
Total_solution,2,solution,1,solution_product,1,
~Kinetic Equations Card
2,
Kinetic_CO2_product,1,CO2_product,1,
   1,KnRc-1,   1.0000,
Kinetic_solution_product,1,solution_product,1,
   1,KnRc-2,   1.0000,
```

Figure 1: Example eSTOMP input card. Each '~' represents the start of a new sub-card which represents a different equation of assumption that need to be specified. Each comma separates different data units that are needed by the sub-cards.

Currently to create these input cards, members of the team will take a pre-created input card and manually go into the text files and edit the variables they want to change. This is not only a slow time-consuming process, but it also requires a deep understanding of eSTOMP and has a high chance of errors which can lead to the waste of important resources. This process also requires them to manually find experimental data when they want to model real life situations. To help improve this process for the team, a way to automate this process was developed and connected to the newly developed grout materials database so that researchers can easily model real life scenarios while also improving efficiency and reducing errors.

Design

For the application to work well with the current workflow of the team the script was developed using python. To make the script as usable as possible a graphical user interface (GUI) was implemented so that any user, no matter what their technical expertise, could create an input card. Once the tools were decided on, a plan to document how the user would interact with the system and how the data would be transferred and modified through the script was created. To easily visualize and document these processes, a Data Flow Diagram (Figure 2) was created to map out how the data would move through the system. The Data Flow Diagram is accompanied by a table listing out the process names and a short description of each process (Table 1).

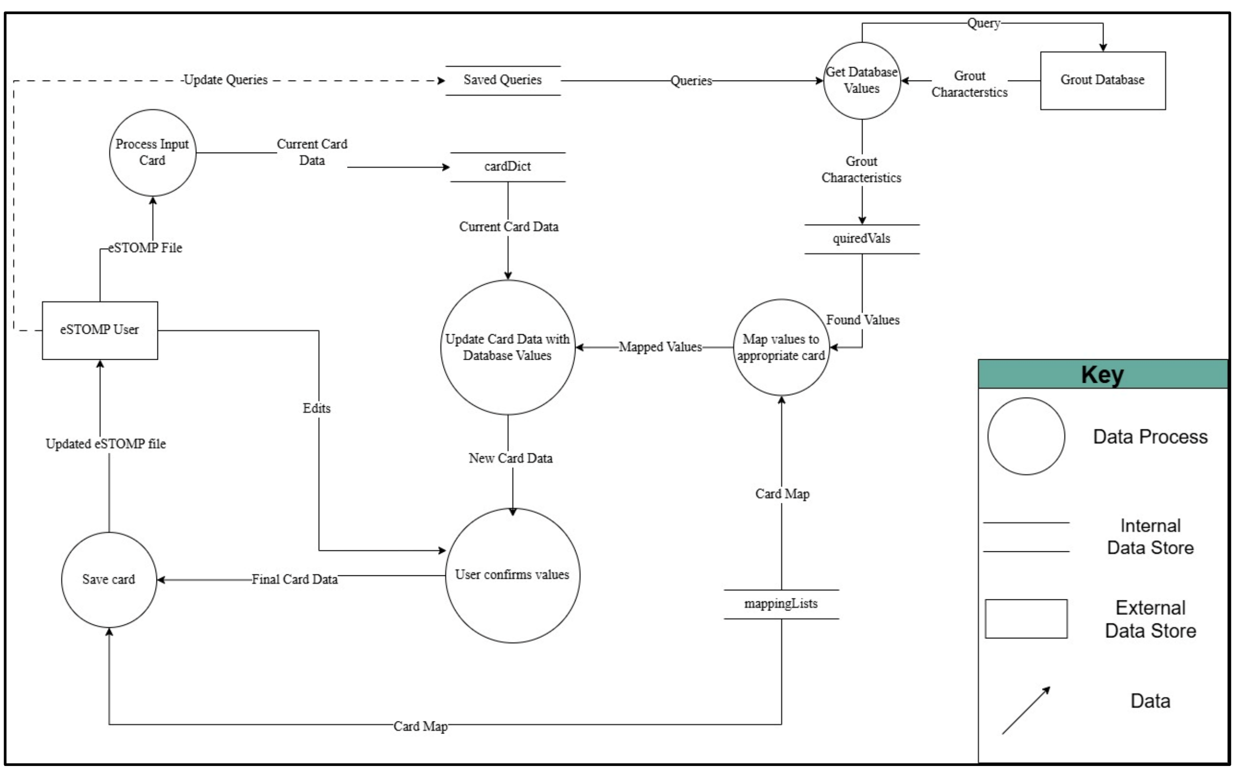

Figure 2: Data flow diagram that shows how data moves through the script and gets transformed.

Table 1:Data Flow Diagram

Process names	Process Descriptions
Process Input Card	• Reads in the users input card file and splits it into its sub-cards. • Saves these sub-cards in a dictionary with the card name as the key and the paragraph as the value.
Get Database Values	• Queries the database for any relevant values and returns what it found in a dictionary.
Map values to appropriate card	• Goes through all the possible variables that could be in each card which is saved in mappingLists and saves what card(s) each database value goes to.
Update card data with database values	• Turns each of the cards found above into a data structure that can be edited. • Finds the correct value to replace in that data structure and updates that value.
User confirms values	• The user can “Zoom in” on any card they are interested in and will see what each value represents and can change those values.

While planning the design of the script, efforts were made to ensure that it would be both user-friendly and scalable for future use. This required a balance between allowing the user to change features via a screen or button in the GUI while not overwhelming them with options and screens they must click through. The design also considers all variations of input cards that users may need to generate. To achieve this, it was necessary to develop a thorough understanding of each sub-card's possible variations and the values it accepts, allowing the script to be designed to accommodate all of them. After gaining familiarity with the eSTOMP card requirements and outlining the design, work on the actual script was started. Through many iterations of testing and development the following script was created.

Results

Following the design outline above this script was created. The script starts by asking the user to input the file they would like to change. Users can either input the card by dragging and dropping or by searching through their directory (Figure 3). The file is then read in by my script and split into each of its sub-cards and saved in a dictionary for future reference. The user is then shown the main input screen of the GUI. Within the main screen, they select how they want to edit their card, what the name of the waste form they're interested in editing is and how many cards they want to create (Figure 4). At this point, the script splits into two separate flows, either the user decides to query the database to get the new values, or they are just interested in editing the card via the GUI. Each of these methods will be discussed separately in the following sections.

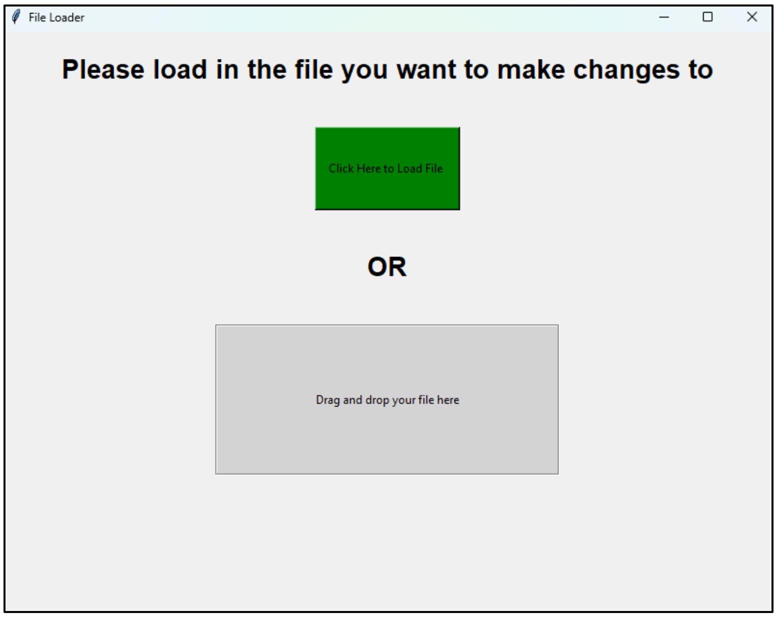

Figure 3: File Loader

Figure 4: Landing Page

Database Query

When the user queries the database, they are first directed to the main screen where they choose how they want to identify the grout they are interested in. They can search by article title, author(s), or sample ID (Figure 5). These search options were chosen so that users could find grout samples from specific papers they were interested in, find papers by certain authors (Figure 6), and search for a specific grout from a paper with its sample ID. Once the user finds a paper they are interested in, they are given a list of all the samples in that paper and relevant information about those samples allowing them to select the grout they are interested in (Figure 7). After a grout is selected, the script moves to a new screen that tells the user what values the query found (Figure 8). If no values are found, the user can either re-query the database, which returns them to the start of the process, or manually edit the values that are in the cards (Figure 9). The user is also able to edit the values that are not found in the query using the same editing screen that is shown in Figure 9. Once the user is happy with the values that they found, they are moved to a final screen where they can check their input card and make any edits to it they want. This gives the user one last chance to check their work and finalize any edits before selecting a file name and saving (Figure 10).

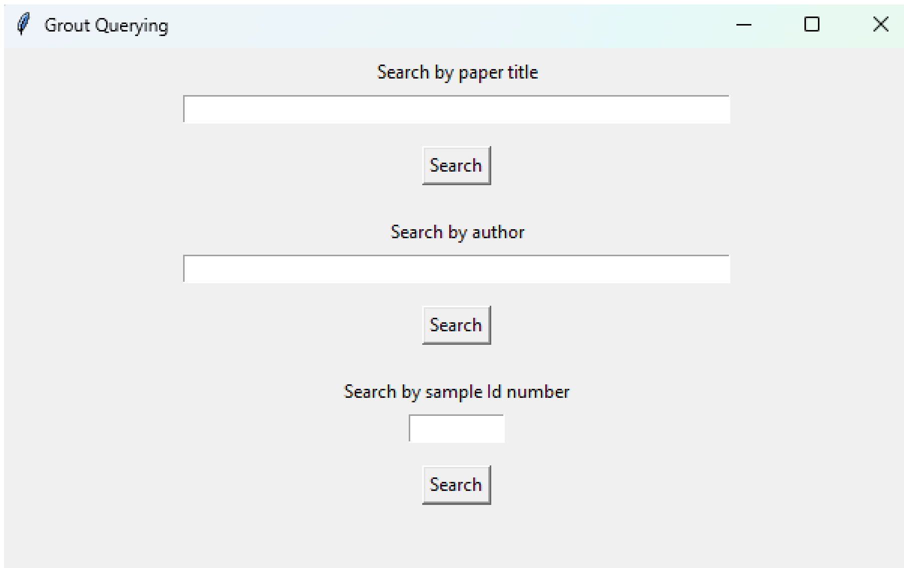

Figure 5: Main Query screen

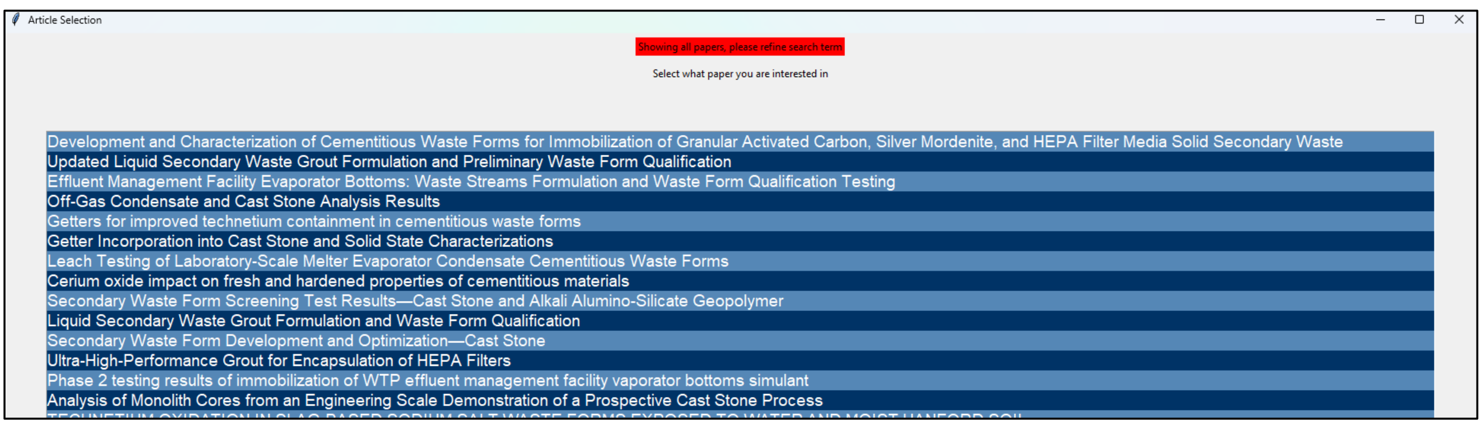

Figure 6: Page that allows you to select what paper you are interested in from the author you searched for

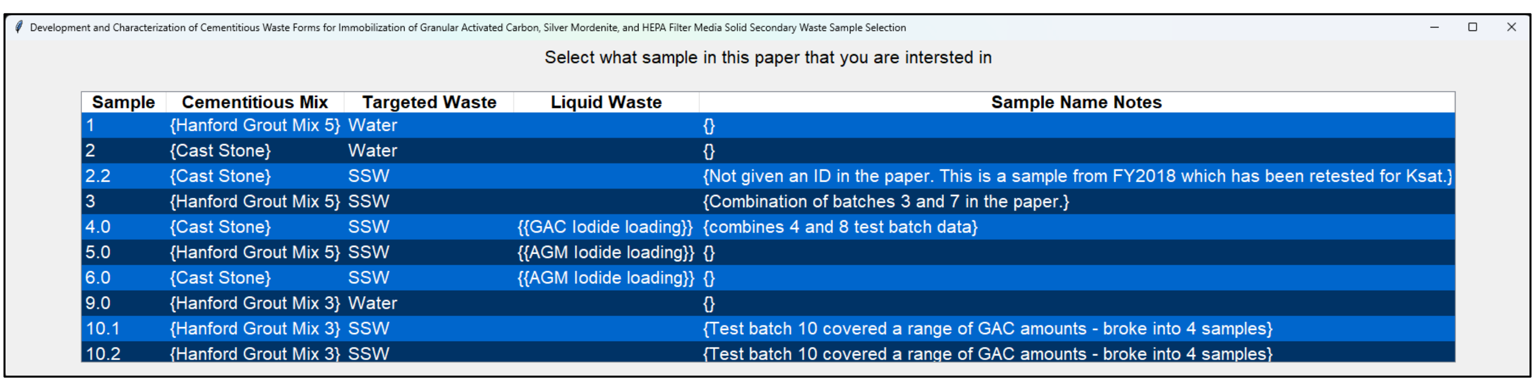

Figure 7: Sample Selection Page

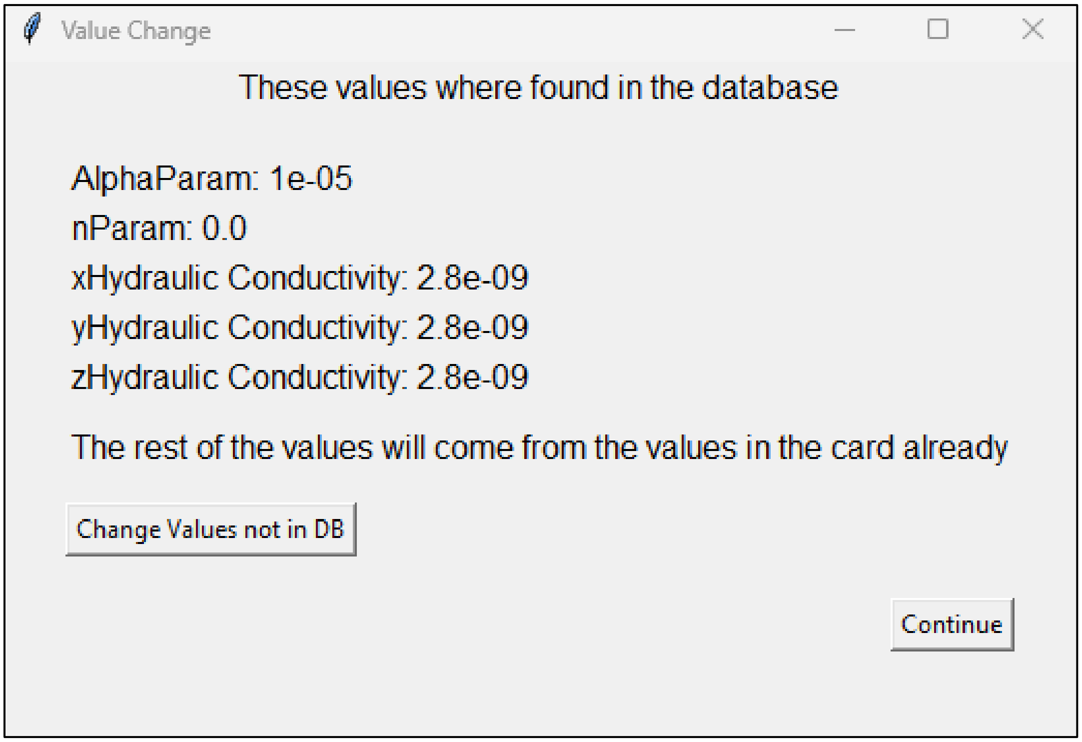

Figure 8: Page that shows what values were returned

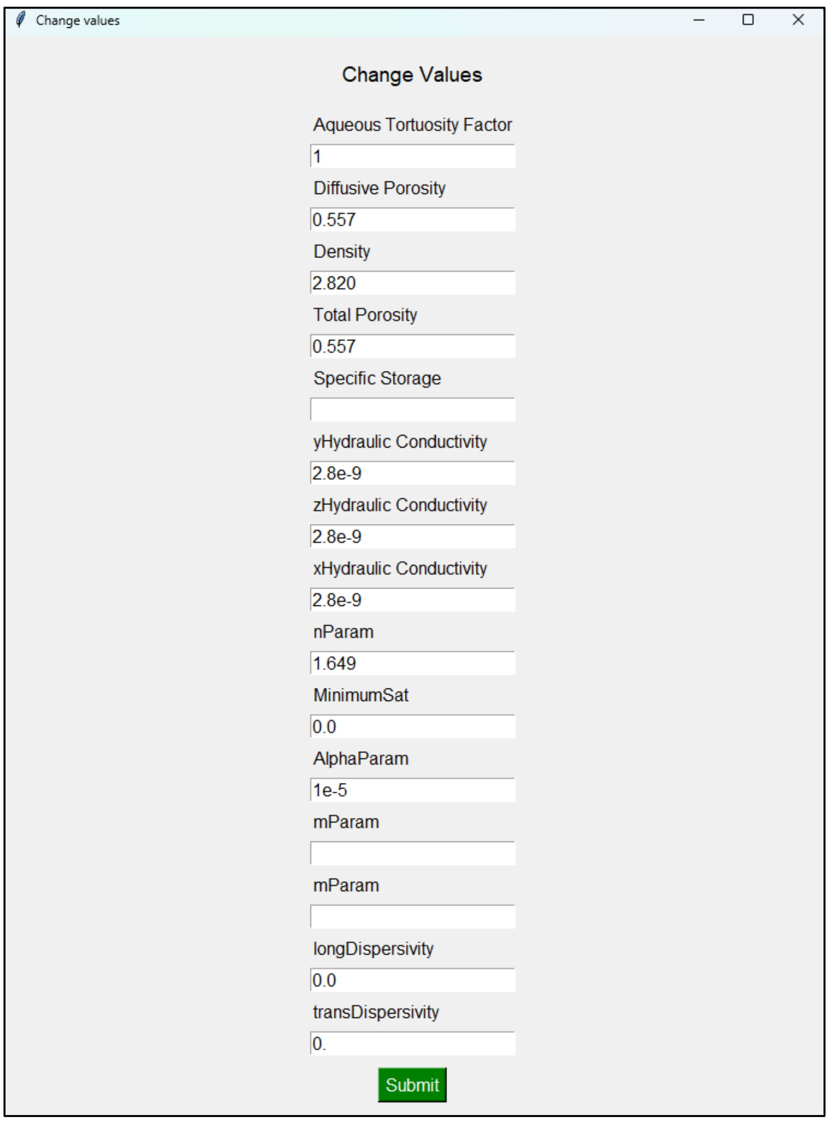

Figure 9: Page that allows you to edit values not found in the database

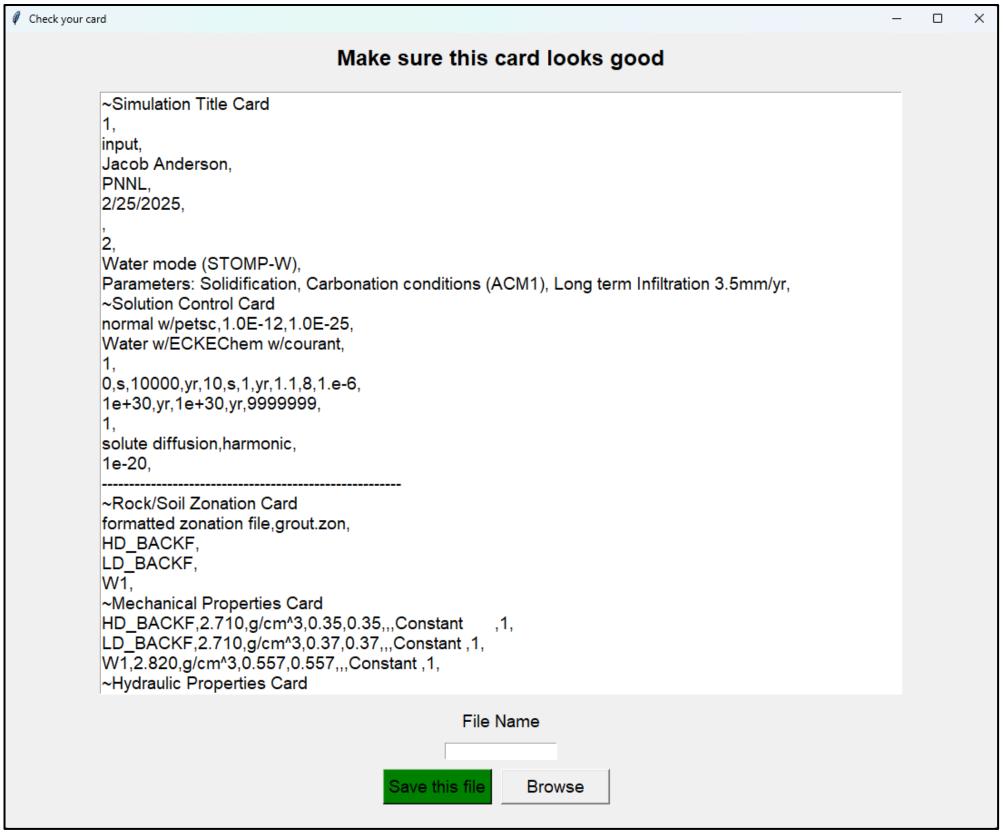

Figure 10: Page that lets you make last edits and save the file

Edit card

When the user decides that they just want to edit the card, they are moved to a screen that gives them a drop-down list of all the sub-cards that were in their input card (Figure 11). When the user selects the card they are interested in, they are moved to a separate screen that will allow them to "zoom in" on that card and edit the values within it. This "zoom" looks different for every card due to the differences in how they are formatted but there are two main ways it works. If it is possible to map what the values in each card represent, the screen will give them a sheet to fill out that has headers describing what each value means and the currently populated values in boxes below each header (Figure 12). This allows the user to easily edit this sub-card without having to have extensive knowledge of how eSTOMP cards are formatted. However, not all cards can be mapped in this way, in these cases a text editing screen comes up that allows you to edit just that sub-card but does not give you any information on what the values mean. This is less user friendly but the cards that follow this design are rarely changed between runs and would likely not be edited by people unless they had a deep understanding of eSTOMP, so this approach still allows them to make edits (Figure 13). Once the user makes all the changes they want within their cards, they submit their changes and are then directed to the save screen as described in the grout query, where they can make any last edits to their card and save it.

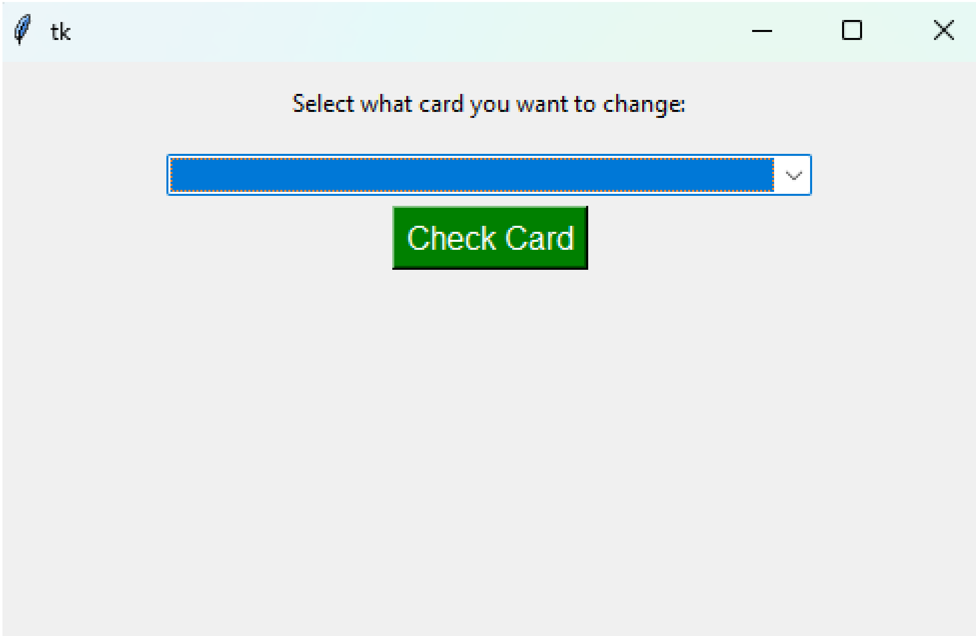

Figure 11: Page that lets you select what card to "zoom in" on

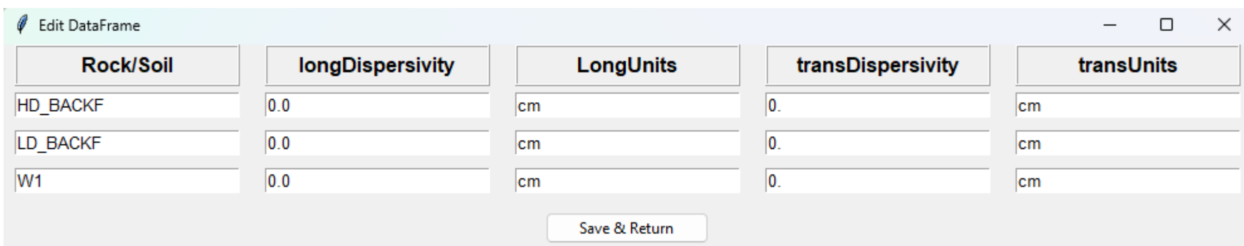

Figure 12: Page that lets you edit the values of the card, with headers describing each value

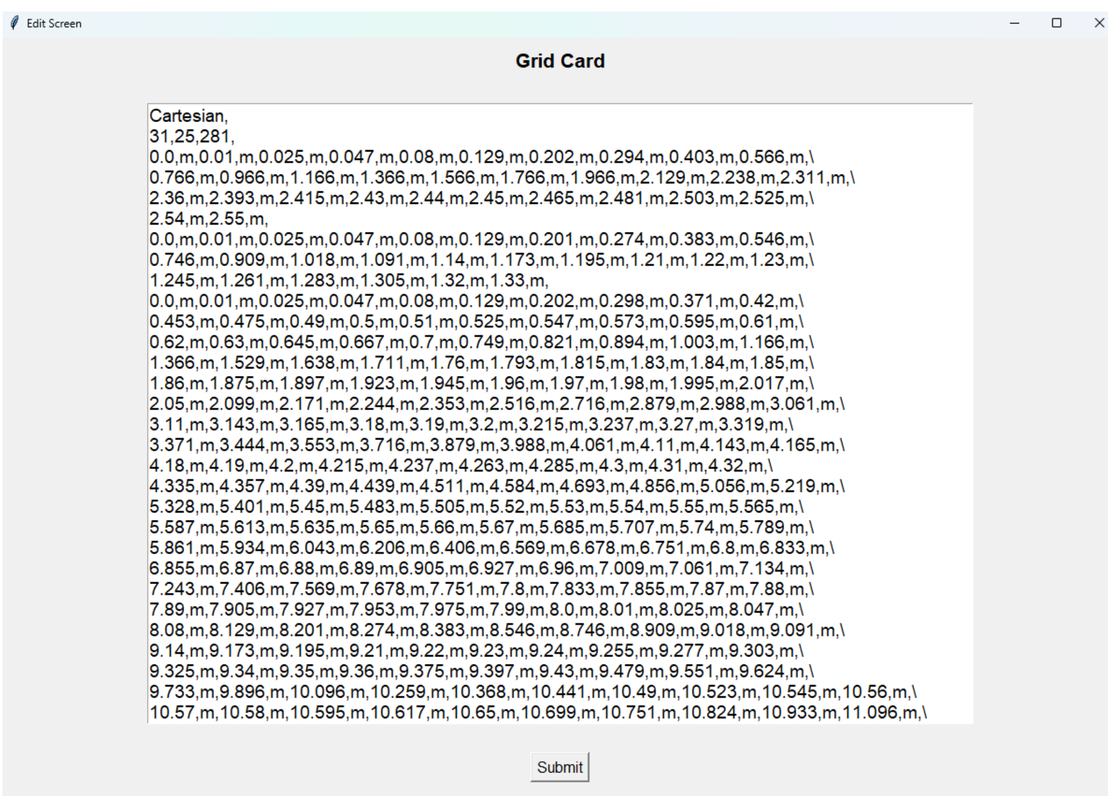

Figure 13: Page that lets you edit the values of the card without headers describing what the values mean

Although there are two main paths that the script follows, there are five clickable buttons that the user sees on the main screen. The two query buttons (batch and individual) give the user a choice in how they want to query the database. If they are creating more than one card, the batch option lets the user select all their grouts from the database and then has them go through and individually save them. In contrast, the individual options allow the user to select a singular grout from the database and save it before returning them back to the query screen to select their next grout sample. This allows users to change their query based on personal preference or the number of input files they are creating. The other option available on the main page (Change Grout and Edit File) combines the two processes described above, allowing the user to query the database and then create edits to their card.

Finally, to ensure that the user has maximum control over what values they are looking for in the database, the last button (Edit Query) allows them to change what values are being queried for. If they decide to change the query, they are directed to a new screen that gives them the current query that is saved within the script (Figure 14). From this screen they can either edit the current variables or select a new one. If they decide to edit a variable, they select one of the currently saved variables and connect it to a new value from the database (Figure 15). If they decide to add a new variable, they must select what card they are interested in and will then get a list of all the variables that eSTOMP allows for that card (Figure 16). Once they select their variable, they then must connect it to a value in the database using the same process as seen in Figure 15. This feature was added so that as the database continues to grow and evolve the user can look for new values or change where certain variables are being searched for without having to manually change the code.

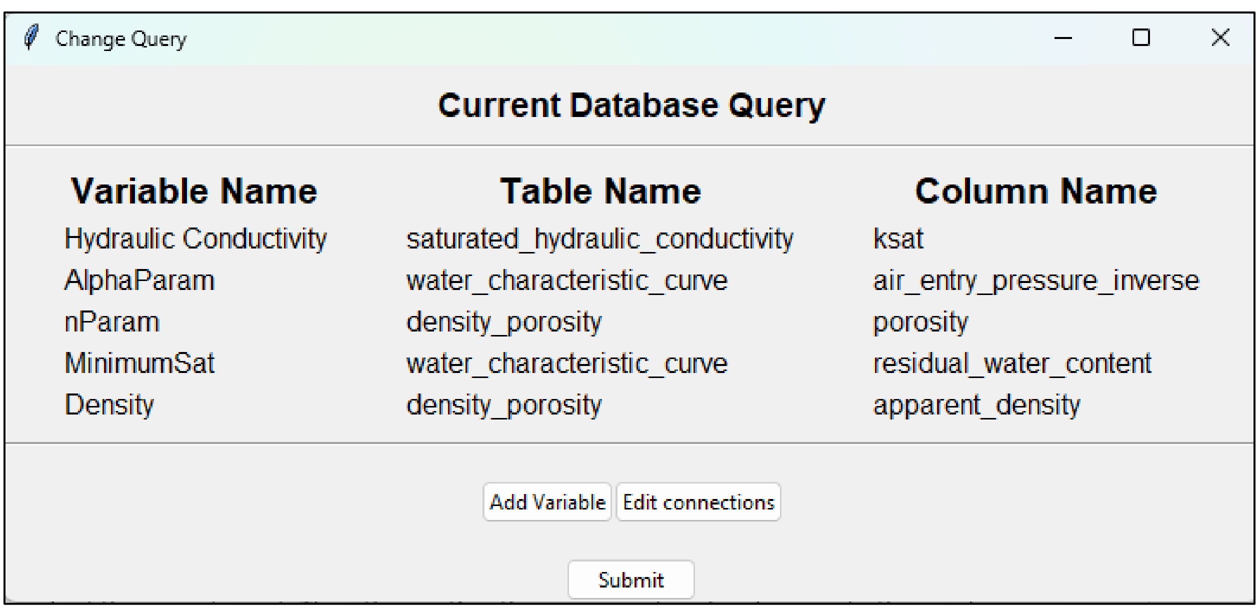

Figure 14: Page that shows you the current connections in the database and allows you to add new variables or edit existing ones

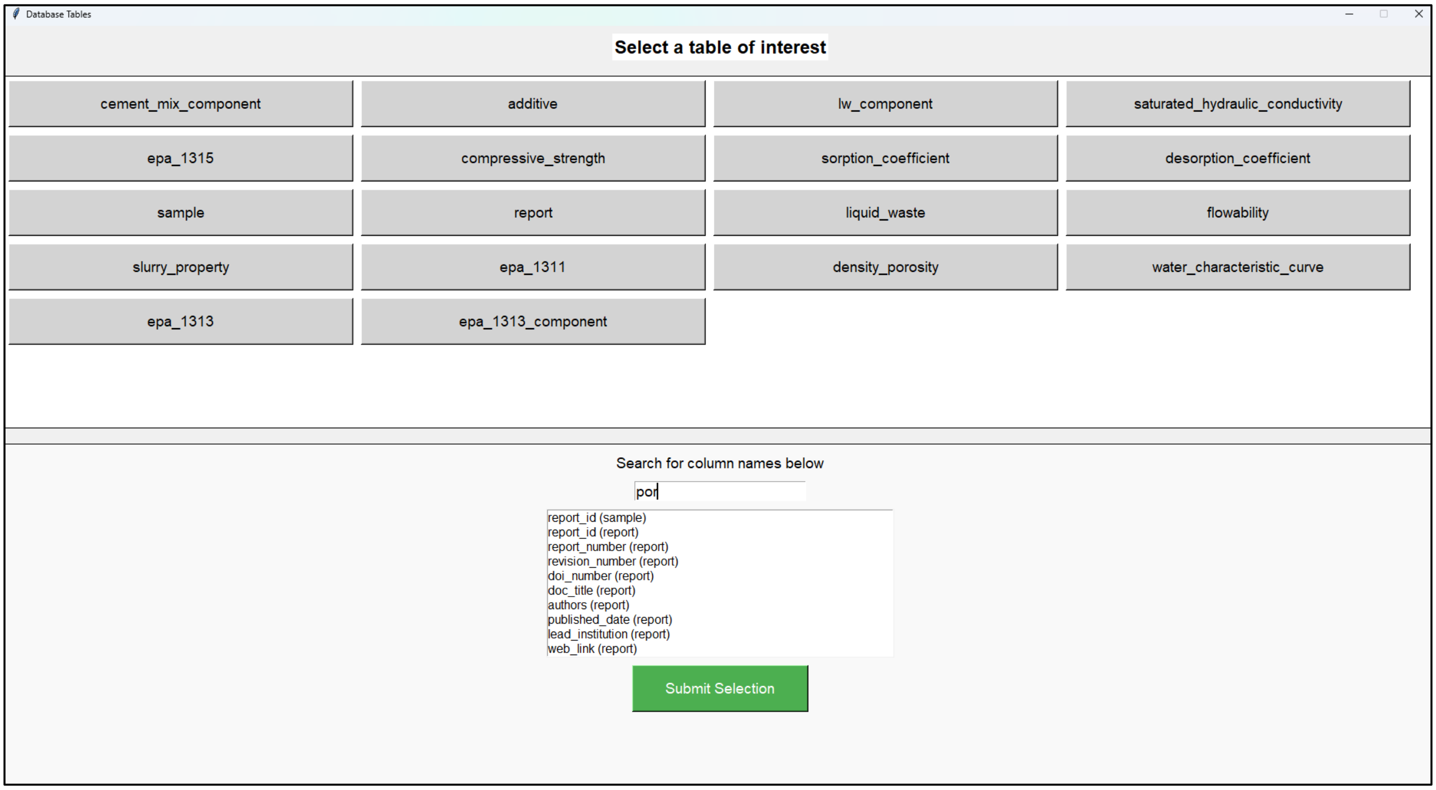

Figure 15: Page that allows you to search the database by table or column name to find what column you want to link with your variable

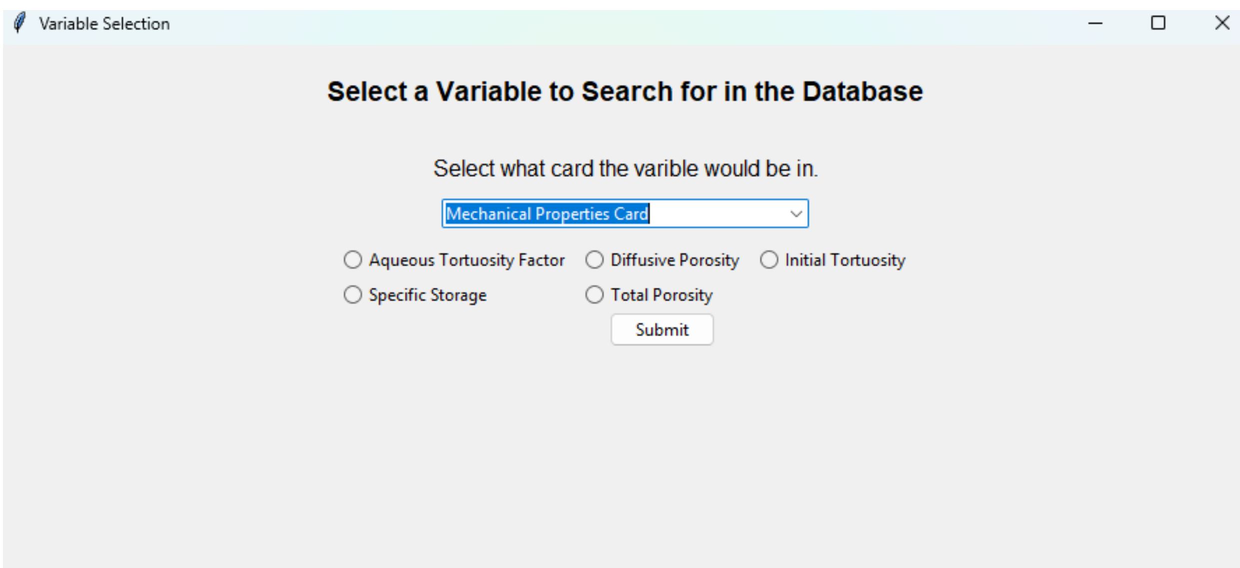

Figure 16: Page that allows you to add a new variable to the current search, will show you all possible variables for the card you are interested in

Validation

To ensure that the script accurately transforms input cards into a way that still falls under the eSTOMP specifications, a validation process was performed by running an eSTOMP simulation with a card generated via the script. To run this simulation, access to the High-Performance Computing (HPC) supercomputer called Constance was required, where the eSTOMP executable is currently stored for team use. Once access to Constance was obtained, an understanding of how to perform a simulation run was needed. To execute the simulation, along with the input card, a job script must be created to provide information such as user identity, output storage locations, and other essential metadata. A job script that was already created was altered, and a pre-made input card from the team was copied to use as a real-world example. The input card was then run through the script, with some grout properties being modified. Once the new input card was created, it was copied back into Constance with the job script and the simulation was run. The simulation produced multiple output files which were postprocessed using a premade perl script and then plotted using Tecplot (Figure 17). This graph serves as a validation that the script successfully executed the simulation, producing output that is both visualizable and ready for analysis by modelers, enabling the extraction of valuable insights.

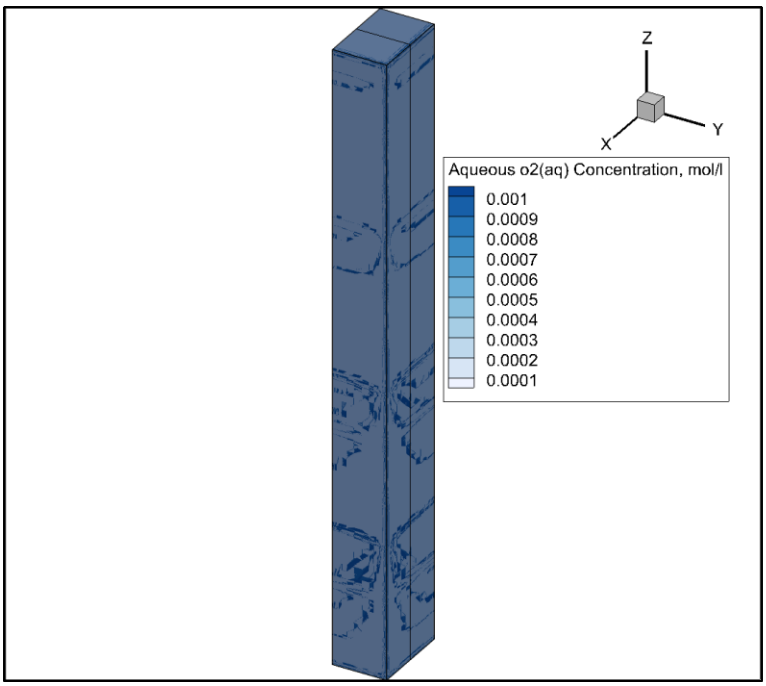

Figure 17: Analysis plot generated with Tecplot for validation that my script works

Conclusions

This script streamlines the process of working with eSTOMP input cards by connecting them to a database of grout properties and enabling users to edit values through an intuitive GUI. The functionality of the script was validated by generating an input card using the tool and running it through an eSTOMP simulation, which produced a graph that confirmed the accuracy of the script's outputs. This tool enhances the efficiency of conducting simulations, saving researchers both time and resources. Additionally, by enabling faster and more frequent simulations, the script allows researchers to explore a wider variety of test scenarios, leading to the creation of models with greater variation and providing deeper insights into their systems. In the future, automation scripts like this can be applied to other aspects of the team's work where data transfer and management are seen as a bottleneck in efficiency. By leveraging automation, researchers can focus less time on repetitive tasks and focus their resources on creating a deeper understanding of their problems so they can come up with unique and effective solutions.

Acknowledgments

This work was supported in part by the U.S. Department of Energy, Office of Science, Office of Workforce Development for Teachers and Scientists (WDTS) under the Science Undergraduate Laboratory Internship Program (SULI). My sincere thanks to Jacob Anderson and Matt Asmussen for their support and mentorship, which this project would not have been possible without. Lastly, thanks to the NNLEMS database team for giving me access to the newly developed grout database and helping me find the data I needed.

References

[1] White, M D and Oostrom, Mart. "STOMP Subsurface Transport Over Multiple Phases Version 3.0 User's Guide.", Jun. 2003.

[2]Asmussen, RM. "Summary of Developing a Hanford Grout Modeling Framework Initial Workshop.", 2024